ELEC40006 Electronics Design Project

Software and Control System Report

EEELunarRover Raw WiFi Production Build

Contents

1 Software and Control System **2**

1.1 Software Requirements Derived from the Rover System 2

1.2 Source-Code Organisation 3

1.3 Main Firmware Loop and Non-Blocking Scheduling 4

1.4 Raw WiFi Networking and HTTP Routing 5

1.5 Dashboard and Operator Control 7

1.6 Mecanum Motor-Control Implementation 8

1.7 Sensor Acquisition and Rock Classification 9

1.7.1 Radio Age Decoding 10

1.7.2 Infrared Pulse-Rate Classification 10

1.7.3 Ultrasonic and Magnetic Classification 11

1.7.4 Multi-Sensor Fusion 11

1.8 JSON Telemetry and Diagnostic Output 12

1.9 Safety and Reliability Features 13

1.10 Testing and Validation Strategy 14

1.11 Limitations and Further Improvements 14

1.12 Quick Interview Reference 15

1 Software and Control System

The software was developed as the integration layer between the rover hardware, the sensing subsystems, and the operator interface. Its role was not simply to run independent sensor code, but to make the complete rover usable during the final demonstration: it had to receive keyboard commands over WiFi, drive a four-wheel mecanum platform, process the radio, infrared, ultrasonic, and magnetic inputs, present rock telemetry to the operator, and stop safely if communication became unreliable.

The final build is organised around a deliberately simple raw HTTP server on the Adafruit Metro M0 and WINC1500 WiFi shield. The browser dashboard sends movement commands to the rover and displays the current classification state. The firmware then separates network handling, sensor processing, motor control, and diagnostic reporting into separate source files so that each part can be tested and explained independently.

Interview reference

The shortest explanation is: the browser sends small HTTP GET requests, `WebRoutes.cpp` parses and dispatches them, `RoverState.cpp` applies motor commands or returns sensor JSON, and the main loop keeps everything non-blocking so driving and sensing continue at the same time.

1.1 Software Requirements Derived from the Rover System

The software requirements were derived from the physical rover design and from the sensing specification. Because the rover uses mecanum wheels, the control code must support sideways translation and rotation as well as simple forward and reverse motion. Because the rock type is defined by a combination of infrared pulse rate, ultrasonic presence, and magnetic polarity, the software must also combine several sensor outputs into a single rock label rather than treating each sensor as a separate result.

ID	Software requirement	Reason	Implemented in
SW1	Provide real-time remote driving over WiFi.	The rover is operated from an external browser during arena testing.	`WebRoutes.cpp`, dashboard HTML
SW2	Support forward, reverse, strafe, diagonal, and rotational motion.	Four-wheel mecanum drive was selected for close-range rock alignment.	`RoverState.cpp` motor functions
SW3	Stop the rover if commands stop arriving.	Loss of WiFi, browser focus, or stale requests must not leave the rover moving.	Dashboard stop logic and firmware watchdog
SW4	Decode the radio age packet received at 600 bps.	Rock age is sent as a four-character ASCII message beginning with `#`.	UART handling in `RoverState.cpp`
SW5	Classify infrared pulse rate as the low or high Poisson source.	Rock type depends on whether the IR pulse rate corresponds to 312 s^{-1} or 547 s^{-1}.	IR interrupt counting and likelihood test
SW6	Read the ultrasonic detector as a binary 40 kHz presence bit.	The analogue ultrasonic circuit already converts the carrier into a digital signal.	`PIN_ULTRASONIC` input
SW7	Read magnetic polarity and reject uncertain values.	Magnetic class depends on the sign of the BMM150 Z-axis reading, but small values are unreliable.	BMM150 polling and dead-zone logic
SW8	Fuse the valid sensor outputs into one rock type.	The operator needs a single classification result, not three unrelated sensor labels.	`lookupRockType()` and `fuseAndClassify()`
SW9	Provide diagnostics without needing recompilation.	Hardware faults are likely during integration and must be separated from WiFi or dashboard faults.	`/debug`, `/i2cscan`, `/api/status`

Table 1: Derived software requirements and their implementation locations.

1.2 Source-Code Organisation

The code is structured as a small set of modules with clear ownership of responsibilities. The most important design decision is that network code does not directly manipulate sensor pins or motor-driver pins. Instead, the web route handler calls public functions from the rover-state module. This reduces coupling and makes it easier to explain faults: if `/motor?cmd=w&speed=255` is acknowledged but the wheels do not move, the HTTP path is working and the likely problem is in motor power, driver wiring, or pin allocation.

File	Main responsibility	Why it matters
`Main sketch (.ino)`	Arduino entry point. Exact source: `EEELunarRover_RawWiFi_Production_motor_fixed.ino`. Runs `setup()` once and `loop()` repeatedly.	Keeps the top-level programme easy to audit: network request, rover tasks, WiFi maintenance.
`AppConfig.h`	Central constants for build mode, timing, pins, sensor thresholds, and motor watchdog interval.	Hardware-facing values are not scattered across the codebase.
`Secrets.h`	WiFi SSID and password placeholders.	Keeps network credentials separate from the main implementation.
`Network.cpp / Network.h`	WINC1500 connection, reconnection, IP reporting, and WiFi status printing.	Isolates WiFi maintenance from HTTP routing and rover logic.
`WebRoutes.cpp / WebRoutes.h`	Raw HTTP parsing, allowlisted routes, dashboard HTML, API responses, and motor command dispatch.	Handles browser communication while keeping sockets short-lived and predictable.
`RoverState.cpp / RoverState.h`	Sensor acquisition, rock classification, motor control, JSON generation, and diagnostics.	Contains the hardware logic and exposes a small interface to the web layer.
`Standalone dashboard HTML`	Development reference. Exact source: `rover_dashboard_integrated.html`.	Useful for interface development, although the deployed production page is embedded in `WebRoutes.cpp`.

Table 2: Software file structure in the production build.

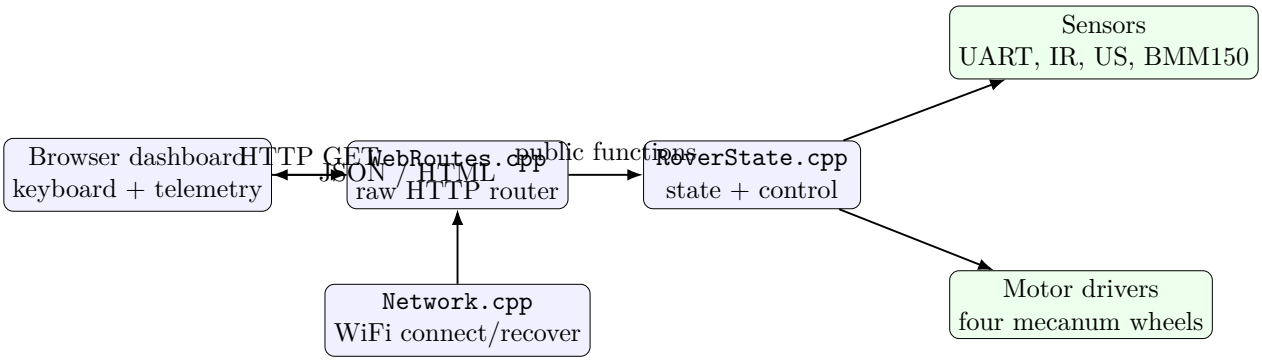


Figure 1: High-level software architecture and data flow.

Interview reference

The architectural point to emphasise is separation of concerns. `WebRoutes.cpp` knows about HTTP routes and query strings, whereas `RoverState.cpp` knows about physical pins, sensor classification, and motor directions. This made integration debugging much faster.

1.3 Main Firmware Loop and Non-Blocking Scheduling

The Arduino sketch follows the standard `setup()` and `loop()` pattern. The setup sequence starts serial debugging, initialises rover hardware, connects to WiFi, and starts the HTTP server. The repeated loop then performs three operations in a deliberate order:

Listing 1: Top-level firmware loop.

```
void loop()
{
  handleHttpClient();
  runRoverTasks();
  maintainWiFi();
}
```

This order gives motor commands high priority because `handleHttpClient()` processes at most one waiting client per pass and then closes the socket. Sensor and watchdog work is handled by `runRoverTasks()`, while WiFi maintenance is periodic rather than constantly blocking the main loop.

Task	Timing constant	Purpose
HTTP client timeout	`HTTP_CLIENT TIMEOUT_MS` 500 ms	Prevents a stalled browser or hotspot socket from blocking later requests.
WiFi maintenance	`WIFI_MAINTAIN INTERVAL_MS` 10000 ms	Checks connection health and restarts the server after reconnection.
Sensor polling	`SENSOR_POLL INTERVAL_MS` 500 ms	Periodically updates IR, ultrasonic, magnetic, and fused rock state.
Serial status printing	`SERIAL_PRINT INTERVAL_MS` 1000 ms	Prints human-readable debug output only when USB Serial is attached.
Motor watchdog	`MOTOR_COMMAND TIMEOUT_MS` 2500 ms	Stops the rover if no fresh movement command is received.
BMM150 retry	`BMM150_RETRY INTERVAL_MS` 2000 ms	Allows the magnetometer to initialise after delayed standalone power-up.

Table 3: Timing constants used to keep the rover responsive.

A key strength of the loop is that sensor updates are based on `millis()` comparisons instead of long blocking delays. This is important because a blocking sensor routine would make the rover slow to respond to a stop command. The code does contain very small `delay(1)` calls when reading or writing HTTP data; these are used only to yield during socket handling and are not used as long control delays.

Interview reference

If asked why the code is responsive, say that the firmware avoids long blocking delays. It services one HTTP request, updates scheduled rover tasks using `millis()`, and only checks WiFi periodically. That keeps motor commands and emergency stops from being delayed by sensor work.

1.4 Raw WiFi Networking and HTTP Routing

The final build uses `WiFiServer` and `WiFiClient` directly rather than relying on a larger web-server wrapper. This choice was made because the raw server path proved more stable with the WINC1500 shield and phone-hotspot style network used during testing. The implementation deliberately reads only the first request line, because the rover does not need to process full HTTP headers to execute

its small set of GET routes.

The lifecycle of one request is as follows:

1. A browser sends a line such as `GET /motor?cmd=w&speed=255&seq=3 HTTP/1.1`.
2. `readHttpLine()` reads the first line within a 500 ms timeout.
3. `parseRequestLine()` extracts the method and target.
4. Non-GET methods and unknown paths are closed without further processing.
5. An allowlisted route is dispatched to the appropriate handler.
6. The response is written with a content length, `Connection: close`, and `Cache-Control: no-store`.
7. The client socket is explicitly closed using `client.stop()`.

Listing 2: Simplified HTTP dispatch pattern.

```
if (strcmp(method, "GET") != 0) {
  client.stop();
  return;
}

if (!isExpectedGetTarget(target)) {
  client.stop();
  return;
}

if (targetMatchesPath(target, "/dashboard")) handleDashboard(client);
else if (targetMatchesPath(target, "/sensor")) handleSensor(client);
else if (targetMatchesPath(target, "/motor")) handleMotor(client, target);
```

Route	Response or action	Interview explanation
`/`	Text landing page.	Confirms that the embedded server is reachable.
`/dashboard`	Production HTML dashboard.	Main operator interface for WASD driving and telemetry.
`/api/ping`	`pong`.	First connectivity test after reset.
`/api/status` and `/api/state`	JSON containing uptime, RSSI, IP address, request count, and motor state.	Shows that the board is alive and the network path is working.
`/sensor`	JSON containing age, IR, ultrasonic, magnetic, rock type, and confidence.	Main telemetry endpoint used by the dashboard.
`/debug`	JSON containing simulated-mode flag, BMM150 status, pin map, and motor state.	Separates wiring or sensor-initialisation faults from web-interface faults.
`/i2cscan`	JSON list of detected I2C addresses and BMM150 status.	Helps diagnose magnetometer power, SDA/SCL wiring, or library initialisation.
`/motor?cmd=...`	Applies a motor command and returns the normalised command.	Lets both the dashboard and manual browser tests drive the rover.
`/health`	`ok`.	Minimal health check requiring almost no payload.
`/favicon.ico`	204 No Content.	Prevents browser favicon requests from appearing as confusing route errors.

Table 4: Allowlisted HTTP routes in the production firmware.

Responses are written in 64-byte chunks. This is a practical reliability decision for a small embedded WiFi server: it avoids one large write monopolising the socket stack and makes dashboard polling more predictable on the limited WINC1500 hardware.

Interview reference

The raw HTTP server is intentionally simple. It is not trying to be a general-purpose web server; it only accepts known GET routes, gives small deterministic responses, and closes every connection so the WINC1500 socket stack cannot become wedged by half-open clients.

1.5 Dashboard and Operator Control

The production dashboard is embedded directly into `WebRoutes.cpp` as a compact HTML, CSS, and JavaScript string. The interface is intentionally minimal: it shows the current command, two-speed PWM state, key indicators, rock telemetry, a small diagnostic section, and buttons for ping, sensor refresh, debug, I2C scan, and motor testing.

The dashboard maintains a JavaScript object called `pressed` containing the movement keys currently held down. On each keydown or keyup event it recomputes a single command string. This avoids the firmware needing to reason about keyboard state; it only receives already-normalised commands such as `w`, `wa`, `q`, or `stop`.

Keyboard input	Command sent	Rover action
W or Arrow Up	w	Forward.
S or Arrow Down	s	Reverse.
A or Arrow Left	a	Strafe left.
D or Arrow Right	d	Strafe right.
W+A	wa	Forward-left diagonal.
W+D	wd	Forward-right diagonal.
S+A	sa	Reverse-left diagonal.
S+D	sd	Reverse-right diagonal.
Q	q	Rotate counter-clockwise.
E	e	Rotate clockwise.
Z	speed toggle	Switches between PWM 110 and PWM 255.
Final key released	stop	Stops immediately.

Table 5: Dashboard keyboard commands and generated firmware commands.

The speed control was deliberately simplified to two values: PWM 110 for controlled low-speed movement and PWM 255 for full speed. This is more useful during a live demonstration than a continuously variable slider, because it gives the operator predictable modes and reduces the chance of accidentally choosing an unusable intermediate value.

A reliability fix was also made in the dashboard request mechanism. Earlier browser builds used temporary image requests to send motor commands. In the production build, motor requests are sent using retained `XMLHttpRequest` objects, which are kept in an array until they finish. This prevents the browser from cancelling or garbage-collecting a request before it reaches the small embedded server. The dashboard displays an acknowledgement including the HTTP status and sequence number after the rover responds.

Interview reference

If asked how the dashboard works, explain that it converts keyboard state into a small command vocabulary. The rover does not receive every key separately; it receives one normalised motion command plus speed. The retained XHR objects and acknowledgements were added because motor control must be more reliable than ordinary dashboard polling.

1.6 Mecanum Motor-Control Implementation

The motor code exposes one public entry point, `applyMotorCommand()`, which accepts a command string and requested PWM speed. It first clamps the speed to the 0–255 PWM range, converts the command to lower case, maps known commands to wheel directions, and stops the rover if the command is unknown. The function also returns the normalised command so the HTTP response can confirm what was actually applied.

At the lower level, each wheel is driven by a direction pin and a PWM pin. The prototype convention is preserved: `LOW` on the direction pin means forward and `HIGH` means reverse. The helper `setAllMotors(fl, fr, rl, rr, speed)` interprets positive values as forward, negative values as reverse, and zero as stopped.

Command	Motion	FL	FR	RL	RR
`w`	Forward	+	+	+	+
`s`	Reverse	-	-	-	-
`a`	Strafe left	-	+	+	-
`d`	Strafe right	+	-	-	+
`wa`	Forward-left	0	+	+	0
`wd`	Forward-right	+	0	0	+
`sa`	Reverse-left	-	0	0	-
`sd`	Reverse-right	0	-	-	0
`q`	Rotate counter-clockwise	-	+	-	+
`e`	Rotate clockwise	+	-	+	-
`stop`	Stop	0	0	0	0

Table 6: Wheel-direction mapping for the four-wheel mecanum drivetrain. Positive means forward, negative means reverse, and zero means no PWM.

Listing 3: Condensed motor command handling.

```
if (c[0] == '\0' || strcmp(c, "stop") == 0) motorStop();
else if (strcmp(c, "w") == 0) motorForward();
else if (strcmp(c, "a") == 0) motorLeft();
else if (strcmp(c, "wa") == 0 || strcmp(c, "aw") == 0) motorFwdLeft();
else if (strcmp(c, "q") == 0) motorRotCCW();
else {
  motorStop();
  known = false;
}
```

The diagonal commands intentionally power only two wheels. This gives simple controlled diagonal motion without requiring the operator to coordinate separate forward and lateral speeds. For close approach to a rock, the most important motion is not maximum speed but small, predictable positional correction.

Interview reference

The mecanum explanation should be linked to the mechanical section: each wheel can contribute a force component in two axes, so changing the sign pattern across the four wheels gives forward, sideways, diagonal, or rotational motion without steering hardware.

1.7 Sensor Acquisition and Rock Classification

The software converts the physical sensor outputs into a single rock classification. The implemented class indices are:

- IR: `0 = LOW` pulse rate, `1 = HIGH` pulse rate.
- Ultrasonic: `0 = ABSENT`, `1 = PRESENT`.
- Magnetic: `0 = UP`, `1 = DOWN`.

These three bits are combined using the project classification matrix. The software packs them into a three-bit code:

$$\text{code} = (\text{IR} \ll 2) \mid (\text{US} \ll 1) \mid \text{MAG}$$

Rock type	IR rate	IR bit	US bit	MAG bit	Software code
Basaltoid	547 s^{-1}	1	1	1	7
Gravion	312 s^{-1}	0	0	1	1
Regolix	312 s^{-1}	0	1	0	2
Lunarite	547 s^{-1}	1	0	0	4

Table 7: Rock-type mapping implemented by `lookupRockType()`.

1.7.1 Radio Age Decoding

The age subsystem receives UART characters over `Serial1` at 600 bps. The parser waits for a `#` start character, then stores the next three characters. A message is accepted only if all three payload characters are digits. For example, `#123` is displayed as an age of 1.23 billion years. Invalid strings are discarded and the previous valid age remains available in the rover state.

Listing 4: Radio age packet validation.

```
if (uartBufferIdx >= 4) {
  uartBuffer[4] = '\0';
  bool valid = true;
  for (int i = 1; i < 4; i++) {
    if (uartBuffer[i] < '0' || uartBuffer[i] > '9') valid = false;
  }
  if (valid) {
    copyText(currentRock.age, sizeof(currentRock.age), uartBuffer);
    currentRock.ageValid = true;
  }
}
```

Interview reference

The radio software does not demodulate the 89 kHz signal directly. The analogue radio circuit converts it to a logic-level UART stream, and the firmware then only has to frame and validate the four-character age message.

1.7.2 Infrared Pulse-Rate Classification

The infrared input is connected to an interrupt-capable pin. Each rising edge increments the volatile counter `irPulseCount`. At the end of the counting window, the software reads and resets this counter with interrupts briefly disabled so that the count cannot change halfway through the read.

The observed rate is

$$\hat{\lambda} = \frac{k}{T},$$

where k is the counted number of pulses and T is the measured window duration in seconds. Rather than using a crude midpoint threshold, the code uses a Poisson log-likelihood ratio between the high-rate and low-rate hypotheses:

$$\log\!\left(\frac{P(k \mid \lambda_H)}{P(k \mid \lambda_L)}\right) = k \log\!\left(\frac{\lambda_H}{\lambda_L}\right) - (\lambda_H - \lambda_L)T,$$

with $\lambda_H = 547\,\mathrm{s}^{-1}$ and $\lambda_L = 312\,\mathrm{s}^{-1}$. A positive log-likelihood ratio gives the high-rate class, while

a negative value gives the low-rate class. The confidence is calculated from the magnitude of the log-likelihood ratio using

$$\text{confidence} = \frac{|\text{LLR}|}{|\text{LLR}| + 5}.$$

This confidence expression is not a formal probability; it is a compact score that increases as the likelihood evidence moves further from the decision boundary.

In real hardware mode, the count window is governed by the elapsed time between sensor-poll calls, with a short-window rejection check below 50 ms. The `IR_WINDOW_MS` constant documents the intended minimum window and is also used directly in simulated mode.

Interview reference

A strong explanation is: the IR code treats photon detections as Poisson counts. It compares which rate, 312 or 547 pulses per second, is more likely to have produced the measured count over the actual time window, then reports both the class and the evidence strength.

1.7.3 Ultrasonic and Magnetic Classification

The ultrasonic analogue chain already produces a 3.3 V digital signal, so the firmware classifies it using a single digital read:

- `HIGH`: 40 kHz signal present.
- `LOW`: 40 kHz signal absent.

A fixed confidence of 0.95 is assigned because this input is already thresholded by the comparator circuit. This confidence is best interpreted as a design weight, not as a measured statistical probability.

The magnetic subsystem uses a DFRobot BMM150 magnetometer over I2C. The code reads the Z-axis magnetic field, stores the value in microtesla, and applies a dead zone of 30 microtesla. If the absolute Z value is inside this dead zone, the magnetic result is marked invalid. Otherwise, the sign gives the polarity:

- $Z > 0$: magnetic UP, class `0`.
- $Z < 0$: magnetic DOWN, class `1`.

The confidence is limited to a maximum of 1.0 and scales with $|Z|/500$. This rewards strong magnetic readings while rejecting weak values near zero. The code also retries BMM150 initialisation every 2 seconds if the sensor is unavailable at boot, which is useful when the rover is powered without USB and the sensor is not immediately ready.

Interview reference

The ultrasonic side is simple because the analogue electronics do the difficult 40 kHz detection. The magnetic side needs more software handling because I2C initialisation can fail at boot and small Z-axis values should not be trusted.

1.7.4 Multi-Sensor Fusion

After each sensor poll, `fuseAndClassify()` updates the three sensor results and determines the rock type. If fewer than two sensors are valid, the type is marked `UNKNOWN`. If all three sensors are valid and the three-bit combination matches the rock matrix, that result is used directly. The fusion confidence is then the average of the three sensor confidences.

If all three sensors are valid but their combination does not correspond to a valid rock type, the code does not immediately discard the result. Instead, it evaluates the valid two-sensor pairs and chooses the pair with the highest average confidence. This is useful because one subsystem can be temporarily wrong or unavailable during integration.

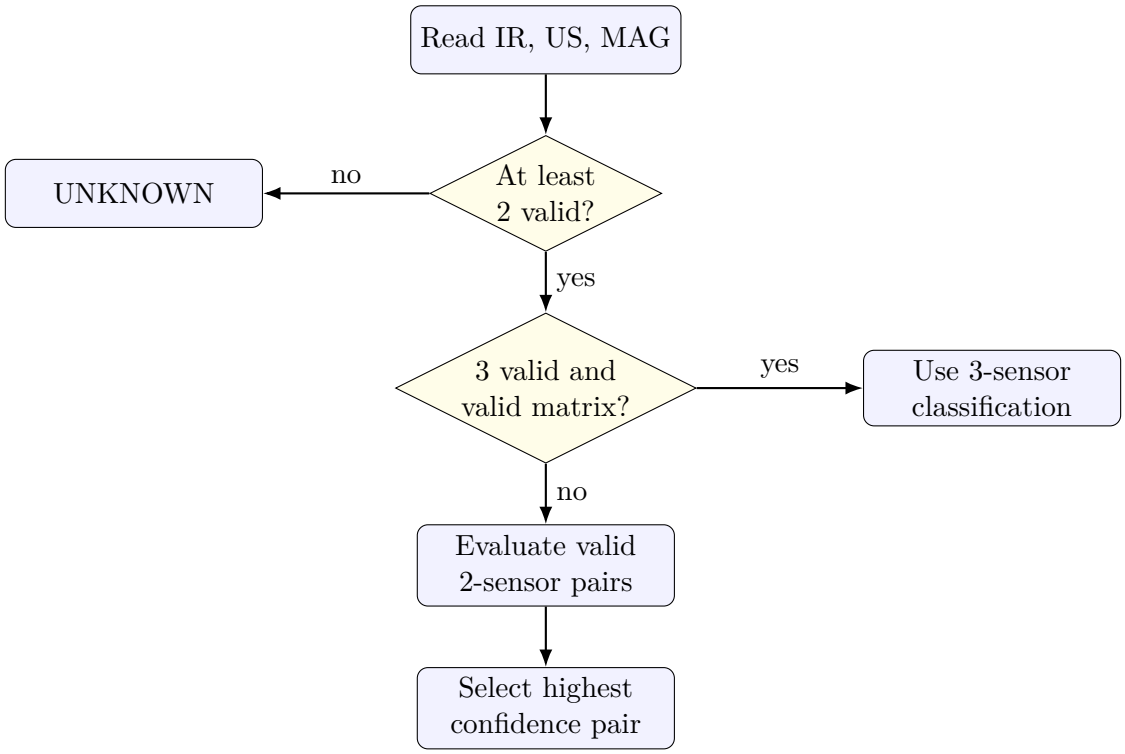


Figure 2: Fusion strategy for valid, missing, and conflicting sensor results.

Interview reference

The fusion logic is fault tolerant. Three agreeing sensors give the strongest result, but the rover can still classify using the best two-sensor pair if one sensor is missing or inconsistent. Fewer than two valid sensors is treated as insufficient evidence.

1.8 JSON Telemetry and Diagnostic Output

The firmware exposes the rover state as JSON because this is easy for the dashboard to parse and also easy to read manually in a browser. The `/sensor` endpoint includes both final outputs and intermediate evidence such as IR count, observed pulse rate, log-likelihood ratio, magnetic Z value, and confidence scores.

Listing 5: Representative structure of the /sensor JSON response.

```
{
  "age":"#123",
  "ir":{"class":1,"conf":0.82,"label":"HIGH","count":110,"lambda":548.0,"llr":22.4},
  "ultrasonic":{"class":1,"conf":0.95,"label":"PRESENT"},
  "magnetic":{"class":1,"conf":0.62,"label":"DOWN","z":310.0},
  "rockType":"Basaltoid",
  "fusionConf":0.80
}
```

This evidence-rich telemetry is useful during an interview because it shows how the final rock label is derived. For example, a Basaltoid result can be explained as high IR rate, ultrasonic presence,

and magnetic down polarity. If the final type is `null` or `UNKNOWN`, the same JSON makes it possible to see which sensor was invalid.

JSON field	Meaning	Use during debugging
`age`	Last valid radio age string, or `null`.	Confirms whether UART age decoding is working.
`ir.label`, `ir.lambda`, `ir.llr`	IR class, measured rate, and likelihood evidence.	Separates weak IR evidence from a confident IR decision.
`ultrasonic.label`	Comparator-derived 40 kHz presence or absence.	Confirms the ultrasonic circuit output reaches the microcontroller.
`magnetic.label`, `magnetic.z`	Magnetic polarity and measured Z-axis field.	Identifies dead-zone or sensor-orientation problems.
`rockType`	Final fused classification.	The value shown to the operator.
`fusionConf`	Average confidence from the selected valid sensors.	Indicates how strongly the current sensor set supports the type.

Table 8: Important fields in the rock-telemetry JSON.

1.9 Safety and Reliability Features

Several layers of protection are implemented because WiFi control can fail in ways that are not present in a wired control system. The design does not rely on a single emergency-stop mechanism; it combines browser-side stop events, command sequencing, keepalive messages, firmware timeout, and safe handling of unknown commands.

Feature	Implementation	Failure mode addressed
Stop on final key release	The dashboard sends `stop` when no movement key remains pressed.	Prevents the rover from continuing after the operator releases the keyboard.
Stop on browser blur or hidden tab	`blur`, `visibilitychange`, and `beforeunload` handlers send `stop`.	Prevents movement if the browser loses focus or closes.
600 ms dashboard keepalive	While moving, the dashboard periodically resends the current motion command.	Keeps the firmware watchdog refreshed only while communication is healthy.
2500 ms firmware watchdog	`enforceMotorWatchdog()` stops if the last command becomes too old.	Stops the rover after WiFi drop-out or dashboard failure.
Sequence number on motor requests	Each dashboard motor request includes an increasing `seq`. Older sequence numbers are ignored.	Prevents a delayed old movement packet from overriding a newer stop command.
Unknown-command stop	Unrecognised motor commands are normalised to `stop`.	Avoids undefined motor states after malformed input.
Allowlisted routes	Unknown paths and non-GET methods are closed immediately.	Reduces unnecessary socket use and keeps routing predictable.
Pin audit at boot	The firmware prints active pins and warns about WINC1500-reserved pins.	Catches wiring conflicts before live testing.
BMM150 retry and I2C scan	The code retries magnetometer initialisation and exposes `/i2cscan`.	Helps diagnose standalone-power and I2C faults.

Table 9: Safety and reliability features in the production build.

Interview reference

A concise safety answer is: the browser tries to stop the rover immediately, but the firmware watchdog is the final authority. If fresh movement commands stop arriving, the microcontroller stops all motors even if the dashboard or WiFi has failed.

1.10 Testing and Validation Strategy

The code was written so that major faults can be isolated using a small number of browser routes and Serial Monitor messages. The recommended test order is important: it avoids blaming sensor or motor code before confirming that the network path is working.

Step	Test	Expected outcome	What it isolates
1	Open Serial Monitor after reset.	IP address is printed and the server starts.	Confirms WiFi credentials, shield presence, and firmware boot.
2	Open `/api/ping`.	Browser displays `pong`.	Confirms basic HTTP connectivity.
3	Open `/api/status`.	JSON contains uptime, RSSI, IP, request count, and motor state.	Confirms JSON response path and network status.
4	Open `/motor?cmd=w&speed=255`, then `/motor?cmd=stop`.	Serial Monitor prints matching motor logs.	Separates WiFi/dashboard issues from motor-driver power or wiring issues.
5	Open `/dashboard`.	Keys update the command display and acknowledgements appear.	Confirms dashboard-to-firmware control loop.
6	Open `/sensor`.	Sensor JSON updates over time.	Confirms sensor acquisition and fusion output.
7	Open `/debug`.	Pin map, simulated-mode flag, BMM150 state, and motor state are reported.	Confirms configuration and internal state.
8	Open `/i2cscan`.	BMM150 address appears when connected and powered.	Confirms I2C electrical connection and magnetometer availability.

Table 10: Recommended software validation sequence.

The same tests also provide an interview explanation route. If a motor command returns HTTP 200 and Serial Monitor prints `[MOTOR] w spd=255`, then the software and network path reached the motor-control function. If the motors still do not turn, the likely fault is outside the web interface: motor supply, H-bridge wiring, common ground, enable pins, or mechanical load.

1.11 Limitations and Further Improvements

The production build is appropriate for a controlled rover demonstration, but it is intentionally compact rather than a full general-purpose robotics framework. The following limitations would be sensible future improvements.

Current limitation	Reason it is acceptable for this project	Possible improvement
No HTTP authentication.	The rover is expected to run on a local controlled network during demonstration.	Add a simple shared token or isolate the hotspot network.
Minimal request parser.	Only a small allowlisted set of GET routes and simple query values are needed.	Add URL decoding and stricter parameter validation if routes become more complex.
Use of Arduino `String` for JSON assembly.	The JSON payloads are small and the code is easier to read.	Replace with fixed buffers if long-duration heap fragmentation becomes a concern.
IR timing constant is clearer in simulated mode than real mode.	Real mode uses measured elapsed time, so the decision remains physically meaningful.	Make the real IR window explicitly configurable and report the chosen window in `/debug`.
Fusion uses the current poll only.	The arena task needs an immediate classification close to each rock.	Add temporal filtering or majority voting across several readings.
Fixed ultrasonic confidence.	The analogue comparator provides only a binary output.	Add pulse-width or envelope-amplitude telemetry if hardware exposes it.
Motor commands are open-loop.	Manual operator control and short arena movements are sufficient.	Add wheel encoders or closed-loop speed control for repeatable trajectories.

Table 11: Software limitations and credible future improvements.

Interview reference

For improvements, avoid saying only that more features could be added. A stronger answer is to link each improvement to a limitation: authentication for network safety, fixed buffers for long-running reliability, temporal filtering for classification stability, and encoders for repeatable mecanum motion.

1.12 Quick Interview Reference

This subsection is designed as a compact recall aid. It gives the wording and technical points that are most likely to be useful if asked to explain the software under interview conditions.

Interview question	Answer to give
What does the software do overall?	It connects the browser dashboard to the rover hardware. The dashboard sends compact HTTP motor commands, the firmware applies them to the mecanum drive, and the same firmware reads the sensors and returns JSON telemetry for rock age, type, and diagnostics.
Why use raw `WiFiServer` rather than a larger web-server library?	The raw path was more stable on the WINC1500 and phone-hotspot setup. It only has to support a few GET routes, so reading the first request line, dispatching allowlisted paths, and closing the client is simpler and more predictable.
How is the code separated?	`Network.cpp` manages WiFi connection, `WebRoutes.cpp` handles HTTP routing and the dashboard, `RoverState.cpp` handles sensors, motor control, fusion, and JSON, and `AppConfig.h` keeps pins and timing constants in one place.

Interview question	Answer to give
How does the main loop stay responsive?	It handles at most one HTTP client, then runs scheduled rover tasks using `millis()`, then periodically maintains WiFi. There are no long blocking sensor delays in the main loop.
How does WASD control become wheel motion?	The browser converts the currently pressed keys into one command string. The firmware maps that string to four wheel signs. For example, forward is `+ + + +`, strafe left is `- + + -`, and rotation uses opposite signs on left and right sides.
How is speed controlled?	The production dashboard has only two PWM values: 110 for controlled movement and 255 for full speed. Pressing `Z` toggles between them and immediately resends the current command if the rover is moving.
How is the rover stopped safely?	The dashboard sends `stop` on key release, browser blur, hidden tab, and unload. It also sends a 600 ms keepalive while moving. If the firmware does not receive a fresh command within 2500 ms, the motor watchdog stops all motors.
What problem does the motor sequence number solve?	Browser or hotspot packets can arrive late. Each dashboard motor request has an increasing `seq`; if an old movement command arrives after a newer stop, the firmware rejects it as stale.
How is radio age decoded?	The analogue radio circuit supplies a 600 bps UART stream. The firmware waits for `#`, accepts the next three digits, and stores a four-character string such as `#123`, representing 1.23 billion years.
How is IR classified?	Rising IR pulses are counted using an interrupt. The code compares the observed count with the Poisson likelihoods for 312 s^{-1} and 547 s^{-1}. The sign of the log-likelihood ratio gives low or high, and its magnitude gives a confidence score.
How are ultrasonic and magnetic readings classified?	Ultrasonic is already digitised by the analogue comparator, so it is read as present or absent. Magnetic uses the BMM150 Z-axis: positive is up, negative is down, and values within the 30 microtesla dead zone are marked invalid.
How does fusion work?	The code first tries the full three-sensor matrix. If all three sensors give a valid rock combination, it uses that. If there is a conflict or one sensor is missing, it chooses the highest-confidence valid two-sensor pair. With fewer than two valid sensors, the type is unknown.
How would you debug no motor movement?	First test `/api/ping`, then `/motor?cmd=w&speed=255`. If Serial Monitor prints the motor command but the wheels do not move, the problem is probably motor supply, H-bridge wiring, common ground, or enable pins rather than WiFi or dashboard code.
How would you debug the magnetometer?	Use `/debug` to see `bmm150_ready` and `/i2cscan` to check whether an I2C address appears. No device suggests power, ground, SDA/SCL, or pull-up issues; a detected device with init failure suggests library or sensor-mode initialisation.

Interview question	**Answer to give**
What is the strongest improvement you would make?	I would add temporal filtering to the classification and explicit authentication or token checking for motor routes. Filtering would stabilise sensor decisions; authentication would make WiFi control safer outside a controlled lab network.

Table 12: Concise interview reference for the software section.